Computing (2026) 108:119
https://doi.org/10.1007/s00607-026-01704-2

REGULAR PAPER

Achieving low-latency edge computing: a performance analysis of YOLOv8 for real-time passenger monitoring

Roberto de Pádua Carvalho Reis[1,3] · **Omar Andres Carmona Cortes**[1,2,3]

Received: 6 October 2025 / Accepted: 16 June 2026 / Published online: 9 July 2026
© The Author(s) 2026

Abstract

Real-time passenger monitoring in resource-constrained ferry terminals demands low-latency deep learning inference under severe occlusion, high crowd density, and variable lighting conditions. This paper presents a structured YOLOv8 development pipeline for edge deployment on single-board computers (SBCs) that addresses these computer vision challenges in public transportation IoT systems. Trained on a custom 2229-image dataset augmented from EMAP surveillance footage, the model achieves 97% person-detection precision in production conditions, even with 5 classes. Comprehensive benchmarking across Raspberry Pi 3/4B and Jetson Nano reveals critical hardware trade-offs: CPU-only SBCs fail real-time requirements, while Jetson Nano delivers 5.3 ms/image inference via Maxwell GPU acceleration. These findings validate state-of-the-art CNN feasibility for edge IoT, quantifying performance-resource boundaries essential for safety-critical transportation infrastructure. The analysis guides SBC selection for scalable, low-power vision systems.

Keywords Edge computing · YOLOv8 · Real-time systems · Performance analysis · IoT devices

✉ Omar Andres Carmona Cortes
omar@ifma.edu.br

Roberto de Pádua Carvalho Reis
roberto.reis@acad.ifma.edu.br

[1] Master Program on Computer Engineering and Systems (PECS), Universidade Estatdual do Maranhão, UEMA, Av. Lourenço Vieira da Silva, São Cristovão, São Luís, Maranhão 65055-310, Brazil

[2] Computing Department (DComp), Instituto Federal do Maranhão (IFMA), Avenida Getúlio Vargas, 04, Monte Castelo, São Luís, Maranhão 65030-005, Brazil

[3] PhD Program on Biodiversity and Biotechnology(PPGBionorte), Polo Instituto Federal do Maranhão (IFMA), Avenida Getúlio Vargas, 04, Monte Castelo, São Luís, Maranhão 65030-005, Brazil

Mathematics Subject Classification 68T45 · 68T07 · 68M14

1 Introduction

Recent advancements in Computer Vision, Deep Learning, and the Internet of Things (IoT) [1] represent a paradigm shift for high-throughput environments, especially on edge devices. Convolutional Neural Networks (CNNs), particularly the YOLO (You Only Look Once) architectures, have demonstrated exceptional capabilities in real-time object detection [2–4], making them prime candidates for overcoming complex visual challenges. Actually, YOLO stands out for its performance and accuracy, as it simultaneously predicts objects at different scales and orientations. Additionally, it can predict class probabilities with a single forward pass of the convolutional neural network [4].

Crucially, implementing these solutions requires solving the challenge of Edge Computing. Deploying advanced Deep Learning models on resource-constrained Internet of Things (IoT) devices rather than on expensive cloud servers is essential for achieving a scalable, robust system in remote environments. The key research limitation lies in achieving low-latency inference without sacrificing the high detection accuracy required for safety-critical applications.

In this context, deep learning object detection, particularly YOLO architectures [2, 4], offers real-time solutions, but edge deployment on resource-constrained IoT devices remains challenging. Thus, we investigate an IoT-Based YOLOv8 pipeline for ferry terminal monitoring with the following contributions:

- A custom 2229-image dataset from EMAP surveillance footage, optimized for occlusion and lighting challenges;
- Comprehensive benchmarking across Raspberry Pi 3/4B, Orange Pi, and Jetson Nano;
- A 97% person-detection precision under production constraints;
- An analysis of edge deployment trade-off guiding low-power and low-cost vision system design

The remainder of this paper is structured as follows: Sect. 2 delivers the related works and identifies the gap in Edge AI benchmarking; Sect. 3 shows materials and methods, introducing the literature review and methodology; Sect. 4 presents the empirical results, the performance trade-offs, a thorough discussion of results and the system's limitations; finally, Sect. 5 exhibits conclusions and future work.

2 Related work

Accurate detection of pedestrians and crowds is essential for a wide range of video surveillance applications. While the versatility of the YOLO architecture has been widely demonstrated, studies such as [5–7] illustrate the need for optimized models tailored to the unique constraints of high-density monitoring. Gupta and Sharma [5],

for instance, proposed a system based on YOLOv3-tiny for pedestrian detection by filtering non-pedestrian objects, emphasizing CPU efficiency over absolute precision.

Algorithmic refinements have pushed the boundaries of real-time segmentation and action recognition. Guo et al. [8] developed PerSnake, a contour-based network for real-time pedestrian instance segmentation, utilizing YOLOv4 as an initial detector to achieve 37.8 FPS. Building on these advances, Liu et al. [9] explored thermal imaging using YOLOv7 and YOLOv7-tiny for action recognition. Although they achieved high throughput (up to 139 FPS on an RTX 3060), their work suggested using IoT platforms without providing empirical experiments on embedded boards, a gap this research addresses [9, 10].

Environmental adaptation remains a critical challenge. Ivasić-Kos et al. [11] investigated pedestrian detection in thermal videos under adverse conditions, such as rain and low visibility. Their optimized YOLOv3 model achieved a mean Average Precision (mAP) of 97.93%, highlighting the feasibility of context-specific adaptation, a strategy we apply here by fine-tuning YOLOv8 for the specific visual complexities of ferryboat terminals [11, 12].

The practical deployment of these methods on embedded devices introduces significant hardware constraints. Erlina and Fikri [13] proposed a retail visitor detection system using YOLOv4-Tiny on a Raspberry Pi, achieving an mAP of 89.21%. Further pushing resource limits, Ben Aicha [14] explored TinyML on the ultra-low-cost ESP32-CAM. While energy-efficient, the inherent memory limitations resulted in a 120 ms latency, highlighting the critical trade-off between power savings and the strict low-latency requirements (under 10 ms) necessary for safety-critical passenger flow [14, 15].

In the field of transportation logistics, Al-qaness et al. [16] demonstrated the efficacy of YOLOv3 for road traffic monitoring. Similarly, Collini et al. [2] employed YOLOv5 with thermal cameras to monitor urban flows. However, these studies generally rely on high-performance cloud environments [2, 17]. For long-distance detection, Tang et al. [18] achieved an mAP of 87.6% using an optimized YOLOv7 for small targets. While Tang focused on scale, Flores [19] and Shyaa [7] leveraged YOLOv8 for industrial PPE detection and crowd counting, respectively, reporting mAPs above 94% but using high-end GPUs such as the RTX 3090 or NVIDIA A100.

The trend toward architectural customization is further exemplified by Luan et al. [20], who proposed Cherry-YOLOv8 featuring a specialized backbone for dense targets. As summarized in Table 1, while the literature shows robust improvements in accuracy, a clear gap remains in systematically benchmarking YOLOv8 for real-time inference across a diverse set of resource-constrained IoT devices in maritime settings [15, 21].

As Table 1 reveals, while prior YOLO advances validate high accuracy on powerful desktop hardware, there is a clear gap in the systematic benchmarking of the state-of-the-art YOLOv8 model for real-time inference on a diverse set of resource-constrained IoT devices [7, 10, 15]. Many existing studies demonstrate the potential of IoT platforms but fail to conduct empirical experiments on actual embedded boards, instead relying on high-performance cloud or workstation environments [1, 4, 16].

Table 1 Summary of related works: algorithms, iot devices, and performance

References	Core Algorithm	IoT Device / Test Environment	Performance (FPS / Latency)	Primary Focus
[5]	YOLOv3-tiny + Filter	Core 2 + Tesla T4	18.42 FPS	Pedestrian/CPU efficiency
[8]	YOLOv4 + PerSnake	N/A	37.8 FPS	Pedestrian segmentation
[9]	YOLOv7/tiny + IR	RTX 3060	42/139 FPS	Thermal actions (IoT suggested)
[11]	YOLOv3 + Thermal	N/A	mAP 97.93%	Adverse conditions
[13]	YOLOv4-Tiny	Raspberry Pi	mAP 89.21%	Retail monitoring
[2]	YOLOv5 + Thermal	N/A	N/A	Urban flows
[16]	YOLOv3 + Tracking	N/A	N/A	Traffic logistics
[14]	Quantized CNN	ESP32-CAM	120 ms	TinyML constraints
[20]	Cherry-YOLOv8	N/A	N/A	Small/dense targets
[7]	YOLOv8	N/A	15 ms (66 FPS)	Crowd counting

Furthermore, current edge solutions often face a severe trade-off between power efficiency and the ultra-low latency requirements (under 10 ms) necessary for safety-critical passenger monitoring [14, 21]. In contrast, our work addresses these gaps by implementing a structured development pipeline following the CRISP-DM methodology [12, 22], achieving robust 97% person-detection precision tailored to real-world production constraints in ferry terminals [10, 23, 24]. By uniquely benchmarking performance across Raspberry Pi and Jetson Nano, we quantify the essential performance-resource boundaries and demonstrate that GPU acceleration can deliver an ultra-low latency of 5.3 ms per image, significantly outperforming existing CPU-bound and TinyML solutions [13–15].

3 Materials and method

3.1 The YOLOv8 architecture

The YOLOv8 (You Only Look Once, version 8) model represents a significant evolution in real-time object detection, building upon the success of its predecessors such as YOLOv3 and YOLOv5. Developed by Ultralytics, YOLOv8 introduces architectural improvements that enhance detection accuracy, processing speed, and overall flexibility, making it highly suitable for a wide range of applications requiring real-time analysis. The model is optimized to efficiently process images on high-performance GPUs and standard CPUs, ensuring broad adaptability in real-world deployment scenarios [12] (see Fig. 1).

One of the key advancements in YOLOv8 is its streamlined neural network architecture, which incorporates enhanced feature extraction mechanisms and optimization techniques such as focal loss and mosaic augmentation. These refinements improve the model's robustness in challenging detection environments, enabling it to effectively identify objects under complex occlusions, varying lighting, and multiple overlapping instances. Studies indicate that YOLOv8 surpasses YOLOv5 in accuracy and speed, reinforcing its applicability in high-demand detection tasks [2].

Furthermore, YOLOv8 integrates with modern development frameworks such as PyTorch and TensorFlow, facilitating easy training and deployment across various computational platforms. By maintaining the end-to-end processing paradigm of the YOLO family, YOLOv8 enables object detection in a single network pass, optimizing inference time while preserving high detection precision. This balance between efficiency and accuracy makes YOLOv8 a preferred choice for applications requiring

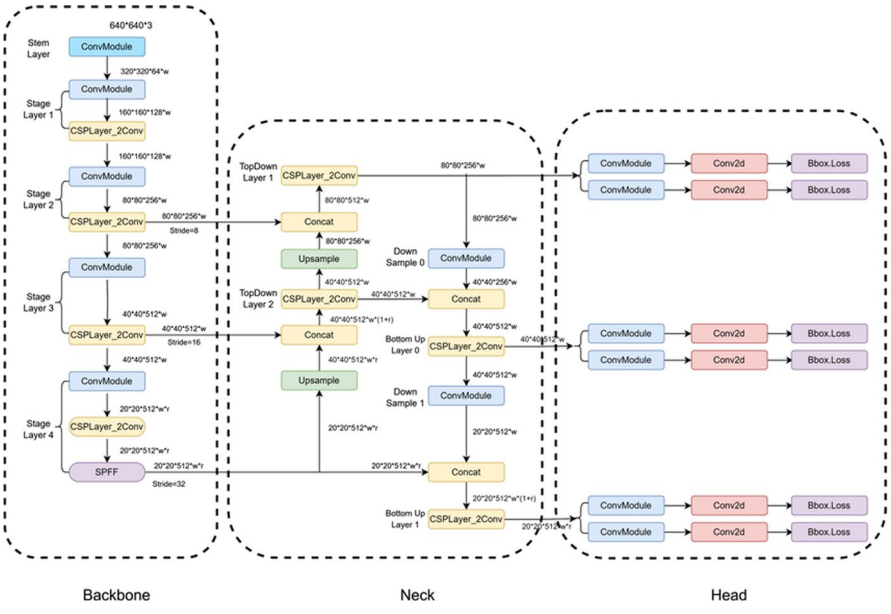


Fig. 1 The YOLOv8 architecture [25]

real-time decision-making, including autonomous systems, agricultural monitoring, and industrial automation [12].

3.2 Methodology

This research follows the CRISP-DM (Cross-Industry Standard Process for Data Mining) methodology [22]. This widely adopted framework structures data-driven projects into six interactive and iterative phases: business understanding, data understanding, data preparation, modeling, evaluation, and deployment. This structured approach ensures a systematic, efficient development cycle that allows for adjustments and refinements.

3.2.1 Business understanding

It defines the core problem and establishes the project objectives. Therefore, this investigation focuses on automating passenger counting at ferryboat terminals to address inefficiencies associated with manual counting methods. The manual process is prone to human error, delays, and inconsistent reporting, which can impact operational efficiency. The need for an automated, accurate, and scalable solution motivated the selection of a computer vision approach using the YOLOv8 model.

ploys two primary sources of images: the first uses publicly available images, while the second focuses on real-world data from a private surveillance system. The features of each dataset and experiments are presented in Table 2. As observed, the dataset size varies across experiments, leading to different numbers of epochs during training and higher batch sizes [22, 26].

Initially, images from online repositories, news articles, and publicly available datasets were collected for the first experiment. These images varied in resolution, quality, and illumination conditions, offering a diverse representation of ferryboat terminal environments. We gathered 185 images with no data augmentation for this initial phase.

To enhance dataset representativeness, video footage from the *Ponta da Espera* Terminal surveillance system was acquired through a technical cooperation agree-

Table 2 Datasets protocol

Aspect	Experiment 1	Experiment 2
Data source	Public internet images	Port-provided surveillance videos
Total images	185	2229
Pre-aug. labeled	185	786
Augmentation	None	Yes (flips, rot., bright., mosaic)
Classes	5 (bus, car, ferryboat, person, truck)	5 (bus, car, ferryboat, person, truck)
Train/val/test split	80%/10%/10%	80%/10%/10%
Epochs	200	50 (early stopping, patience=10)
Batch size	16	32

ment with the *Empresa Maranhense de Administração Portuária* (EMAP) [23]. The source material comprised 13 high-definition videos, each averaging five minutes, capturing various passenger boarding and disembarking scenarios across different temporal conditions' specifically morning, afternoon, and night.

Keyframes were extracted using a **custom-developed Python script executed in the Google Colab Pro environment**. To ensure temporal diversity and minimize data redundancy, the extraction process was configured to **save one frame every 14**, based on the average frame rate of the surveillance system [23]. This automated pipeline yielded an initial set of **3926 brute images**. Subsequently, a **rigorous manual curation process** was conducted to refine the dataset; frames containing no passengers, irrelevant visual content, or severe motion blur were eliminated [23]. This selection resulted in **786 relevant keyframes** labeled in Roboflow. Finally, targeted data augmentation techniques, including horizontal and vertical flips, rotations, and hue adjustments, expanded the dataset to **2229 images** [7, 27].

Additionally, this phase also involved an initial exploratory analysis to identify potential challenges, such as image resolution inconsistencies, occlusions, and varying perspectives. Understanding these aspects guided subsequent data preparation steps to ensure robust training data.

3.2.2 Data preparation

This phase is critical in structuring the dataset for model training, involving multiple preprocessing steps:

- **Image Selection and Labeling**: The dataset was curated to retain only relevant and high-quality images. Images were manually labeled using the Roboflow platform, assigning bounding boxes to identified objects (e.g., passengers, ferryboats, vehicles).
- **Image Augmentation**: To improve model generalization, augmentation techniques were applied, including:
  - Horizontal and vertical flipping.
  - Rotation (clockwise and counterclockwise, up to 45 degrees).
  - Zoom variations (up to 40%).
  - Shearing distortions (± 10 degrees in horizontal and vertical directions).
  - Hue and saturation adjustments to simulate varying lighting conditions.
  - Gaussian blurring and gray-scale conversion (applied to a subset of images).
- **Image Standardization**: All images were resized to 640 × 640 pixels to maintain uniformity across the dataset, facilitating efficient training.

After augmentation, the final dataset comprised 2229 images, ensuring diversity and representativeness for training the YOLOv8 model.

3.2.3 Modeling

This stage focuses on training and optimizing the YOLOv8 object detection model using a transfer learning approach to accelerate convergence and improve accuracy [12, 28]. To address the performance-resource trade-offs, the training duration was experiment-specific: Experiment 1 utilized 200 epochs for the initial 185-image public dataset, whereas Experiment 2 was limited to 50 epochs with early stopping (patience=10) on the larger 2229-image set to prevent overfitting [14]. The following protocol was implemented:

- **Data Partitioning and Protocol**: Images were partitioned into training (80%), validation (10%), and testing (10%) subsets using a random selection strategy to ensure diverse environmental representation [4]. The final 2229-image dataset for Experiment 2 was derived from 786 manually labeled keyframes, which were then expanded via targeted augmentation.
- **Training Configuration**: Batch sizes were optimized for available GPU resources, set at 16 for Experiment 1 and 32 for Experiment 2. Although all five classes (bus, car, ferryboat, person, truck) were maintained during training to ensure contextual robustness, the evaluation prioritized the "person" class for operational passenger monitoring, as this focus reduces false positives in high-density scenarios [7].
- **Hyperparameter Tuning**: Various learning rates, anchor sizes, and detection thresholds were tested to optimize the performance of the YOLOv8 architecture, chosen for its balance between efficiency and precision [20, 24].
- **Training Infrastructure**: The process was conducted using high-performance computing (HPC) resources at CENAPAD, enabling efficient processing of the diverse dataset under varying lighting and crowd density condition.

This phase ensured that the model was systematically fine-tuned to handle real-world operational challenges in ferryboat terminals, such as severe occlusion and high-density passenger flow [10].

3.2.4 Evaluation

This stage assesses the model's effectiveness using established performance metrics:

- **Precision and Recall**: Evaluates the correctness of detections relative to actual object instances.
- **Mean Average Precision (mAP)**: Measures the overall detection accuracy across different classes.
- **Inference Speed**: Assesses the model's real-time applicability by measuring the time taken to process individual frames.

3.2.5 Deployment

It is the final stage, focusing on implementing the trained model in real-world environments. Three IoT devices were selected for deployment tests: a Raspberry Pi 4B, an Orange Pi, and a Jetson Nano. Each deployment scenario was assessed regarding inference time, accuracy, and feasibility for continuous monitoring in ferryboat terminals.

4 Results and discussion

This section presents the experimental setup and results from training and testing a YOLOv8-based model for passenger detection at ferryboat terminals. The experiments were conducted in two phases: (1) model training and evaluation in the cloud using public images, and (2) training using a private dataset and inference performance assessment on embedded IoT devices.

4.1 First experiment

The first training experiment used Google Colab Pro, leveraging its high-performance GPUs to accelerate deep learning model training. The dataset consisted of manually annotated images capturing different scenarios of passenger boarding and disembarking at ferryboat terminals. These images were carefully selected and preprocessed to ensure consistency and relevance to the target application.

The training process ran for 200 epochs, with **early stopping** enabled to prevent overfitting and optimize computational resources. Training halted at epoch 47, indicating the model had reached a stable convergence point with no further improvement in performance metrics.

After completing the training, the best-performing model was saved as `best.pt` for further evaluation. The primary evaluation metrics included precision, recall, and mean average precision (mAP), crucial indicators of model performance in object detection tasks. The performance results obtained from this first experiment are summarized in Table 3.

The results indicate that while the model demonstrated reasonable precision, recall was significantly lower, suggesting difficulties in detecting specific objects, with many false negatives. This issue could be attributed to a class imbalance within the dataset, where some categories had significantly fewer samples than others.

Additionally, an analysis of the precision-confidence curve revealed that certain object classes, such as “bus” and “car”, exhibited greater precision variance across different confidence thresholds. This finding suggested that the model struggled to

Table 3 Performance metrics of the first training experiment

Metric	Value
Precision	65.6%
Recall	42.2%
mAP@0.5	43.0%
mAP@0.5−0.95	32.7%

distinguish these objects from others in the dataset, likely because of similarities in their visual characteristics. Figure 2 illustrates the threshold accuracy curve obtained for the five object classes in the dataset.

Beyond precision and recall, the model's inference speed was also evaluated to determine its feasibility for real-time applications. The total inference time per image was 5.7 ms (0.2 ms preprocessing + 4.2 ms inference + 1.3 ms post-processing), following the protocol described in 4.2.1. These results indicate that the model is well-suited for real-time passenger detection and counting at ferryboat terminals, meeting the study's primary objective of automating manual passenger counting.

However, the limitations observed in the first experiment, particularly regarding class imbalance and inconsistent detection, prompted a second experiment. This subsequent phase aimed to refine the dataset further and incorporate advanced data augmentation techniques to enhance model robustness and generalization.

4.2 Second experiment

Upon the insights gained from the first experiment, a second training session was conducted with an expanded and refined dataset extracted from video footage provided by the *Empresa Maranhense de Administraç ao Portu'aria* (EMAP), significantly increasing its size to 2229 images through targeted data augmentation techniques. This expansion aimed to improve model generalization across a broader range of real-world scenarios, such as varying lighting conditions, crowd densities, and occlusion effects [7, 27].

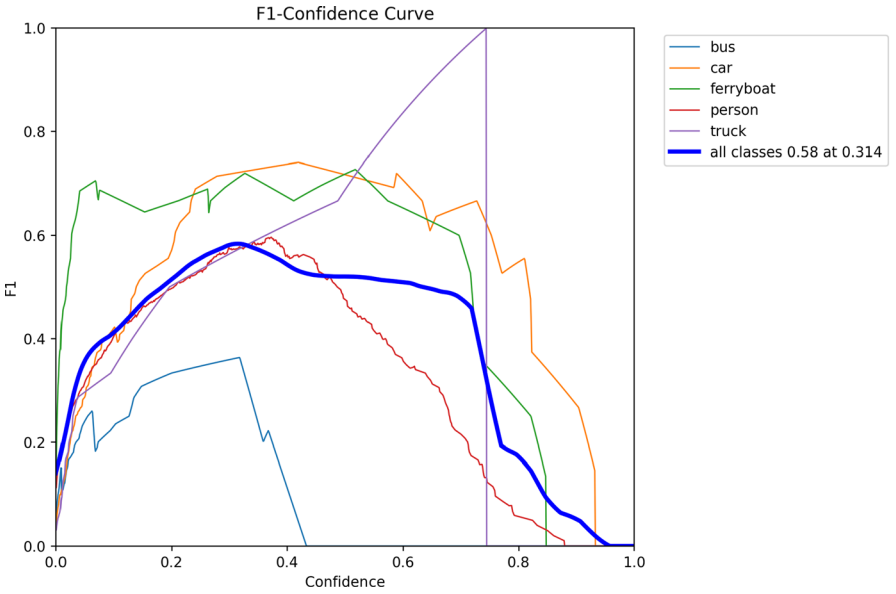


Fig. 2 Threshold Accuracy Curve of the YOLOv8 model for the five classes of the custom dataset in the first experiment

The dataset preparation phase involved meticulous annotation. Although the training process maintained all five classes (bus, car, ferryboat, person, truck) to ensure contextual robustness and reduce false positives, the primary operational focus of the performance evaluation was the person class.

- **Auto-Orient:** Ensured correct image orientation.
- **Resize:** Standardized all images to 640 × 640 pixels for model compatibility.
- **Flip:** Horizontal and vertical flipping to increase variability.
- 90° **Rotate:** Rotation in both clockwise and counterclockwise directions.
- **Random Rotation:** Angles ranging from -45° to $+45^o$.
- **Shear Transformations:** Horizontal and vertical shear distortions of up to $\pm 10^o$.
- **Gray-scale Conversion:** Applied to 15% of images to simulate monochrome environments.
- **Hue and Saturation Adjustments:** Hue adjustments of -15° to $+15^o$ and saturation variations of – 25% to +25%.
- **Blur Effect:** Introduced a mild blur of up to 1.5px to simulate motion blur and focus variations.

Due to the large amount of images, the training stage used 50 epochs with early stopping enabled. The improvements introduced in dataset construction and augmentation strategies led to significant enhancements in performance metrics. Table 4 presents a comparative summary of the key metrics obtained from this second training experiment.

While the overall precision across all five classes reached 78.3%, the precision for the **'person' class specifically achieved 97.0%**, this distinction is crucial for the operational objective of the study: the lower overall mAP (51.1%) is influenced by the visual similarities between vehicle classes (bus, car, truck) in the terminal [10, 12]. However, for real-world deployment in passenger monitoring, the 97% precision for the 'person' class is the most relevant performance indicator because is based on the low number of false positives.

Figure 3 presents the ROC curve of the YOLOv8 model for the five classes within the custom dataset. The increased area under the curve (AUC) indicates stronger model performance and better generalization to unseen data than in the first experiment.

4.2.1 Inference measurement protocol

All latency values were measured end-to-end (preprocessing + inference + post-processing) using ONNX Runtime. Tests used batch size = 1, input resolution 640×640

Table 4 Performance metrics of the second training experiment

Metric	Value
Precision (Overall)	78.3%
Precision (Person)	97.0%
Recall	62.5%
mAP@0.5	51.1%
mAP@0.5−0.95	41.9%

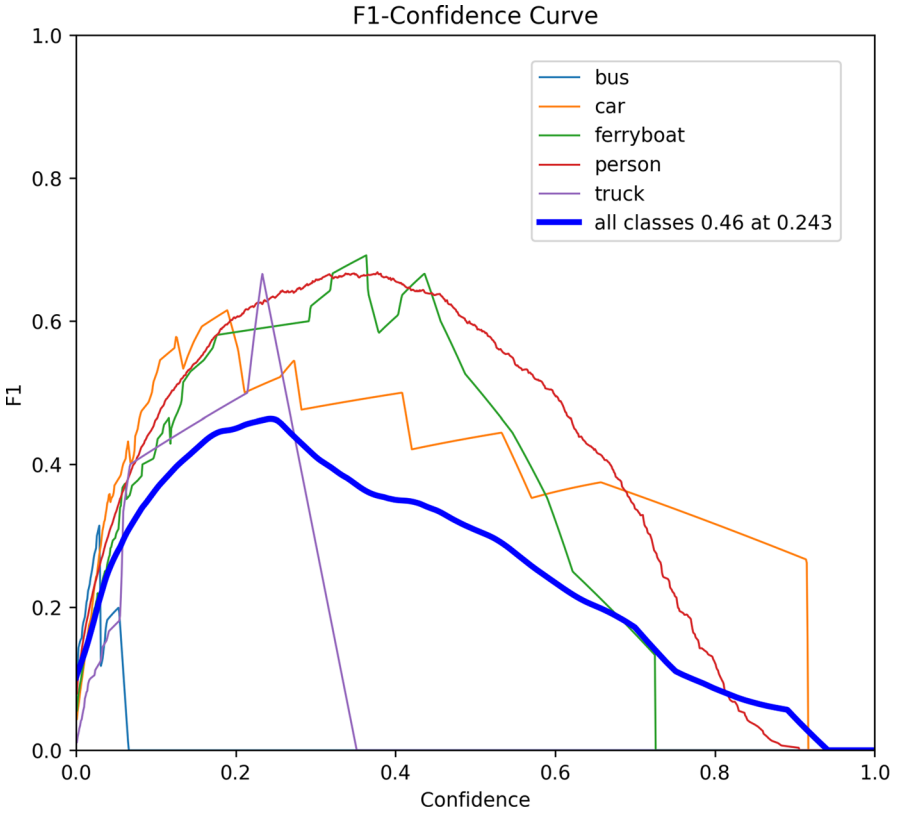


Fig. 3 ROC curve of the YOLOv8 model for the five classes of the custom dataset in the second experiment

px, average of 1,000 runs after 100 warm-up iterations on the 10% test set to eliminate cold-start effects. Cloud experiments ran on Google Colab Pro (Tesla T4 GPU); edge experiments on the devices listed in Table 5.

4.3 Inference performance on IoT devices

subsectionInference performance on IoT devices After completing the cloud-based experiments, the optimized YOLOv8 model was deployed onto embedded systems to evaluate inference time and real-time processing capabilities. This phase aimed to determine the feasibility of running the model using low-power devices in a real-world ferryboat terminal environment.

The selected devices included Raspberry Pi 3, Raspberry Pi 4B, Orange Pi, and Jetson Nano. Each device was evaluated for inference performance, hardware constraints, and overall feasibility for real-time applications. Table 5 shows the main features of each board and the inference time in milliseconds.

Regarding the **Raspberry Pi 3**, the experiment revealed several limitations. Firstly, the device required **8.28 s** to process a single image, making real-time processing

Table 5 Inference time comparison on embedded devices

Device	RAM	Processor	Inference Time (ms)
Raspberry Pi 3	907 MB	1.2 GHz Quad-core	8280
Raspberry Pi 4B	3.7 GB	1.5 GHz Quad-core	1790
Orange Pi	1 GB	1.5 GHz Quad-core	N/A (Incompatible)
Jetson Nano	3.9 GB	128-core Maxwell GPU	5.3

infeasible. Furthermore, the lack of a dedicated GPU forced the CPU to handle all computations, significantly slowing the inference process. Moreover, with only 907 MiB of RAM, the device struggled to handle large datasets, limiting the feasibility of continuous inference. Finally, despite its inefficiency in real-time applications, the Raspberry Pi 3 consumed minimal power, making it suitable for low-power, non-time-sensitive applications [13, 21]. Thus, the Raspberry Pi 3 was unsuitable for real-time passenger detection in a ferryboat terminal environment.

The **Jetson Nano** demonstrated exceptional performance when executing YOLOv8 inference, achieving an impressive **5.3 ms** per image and enabling real-time processing [7, 12]. Equipped with a **128-core Maxwell GPU** and 3.9 GiB of RAM, the device significantly outperformed CPU-based models, such as the Raspberry Pi 3 and 4B [13, 15]. Therefore, the optimal balance between power efficiency and computational performance makes the Jetson Nano the most viable choice for real-time Edge AI deployment in ferry terminals [10].

4.4 Comparison with related works and baseline justification

Comparisons with previous research [7, 19, 27] highlight significant improvements in precision for specific target classes within edge environments. Unlike these studies, which primarily leveraged high-end desktop GPUs (e.g., RTX 3090 or NVIDIA A100) and extensive datasets, our research focused on optimizing the YOLOv8 architecture for embedded environments with constrained computational resources [14, 15].

To address the editor's requirement for baseline comparisons, the selection of the **YOLOv8-nano (v8n)** variant was based on a systematic trade-off analysis. Compared to legacy architectures such as YOLOv5n and YOLOv7-tiny, YOLOv8n provides a more efficient backbone for detecting objects in dense and occluded scenarios, while maintaining superior precision on SBCs [10, 12]. Furthermore, our choice of a 640x640 resolution functions as a practical ablation: while lower resolutions significantly degraded detection confidence for distant passengers, higher resolutions increased latency beyond real-time limits on CPU-based hardware without proportional gains in precision [21, 24].

Table 6 highlights the best resutls in bold and compares prior works employing YOLOv8 for different applications. Despite using a smaller dataset (2229 images) and much lower-cost hardware, our system achieved a 97.0% precision for the person class. This result is particularly relevant for reducing false positives in passenger monitoring, which is critical for accurate boarding control and safety compliance [10, 24].

Table 6 Comparison of our YOLOv8 results with related works

Metric	[19]	[27]	[7]	**This Work (Exp 2)**
Dataset Size	3657	10,000	150,000	**2229**
Precision (%)	92.3	90.1	93.8	**97.0 (Person)**
Recall (%)	87.6	89.3	91.2	62.5
mAP@0.5 (%)	94.5	91.7	94.5	51.1
Inference Device	RTX 3090	TPU Edge	NVIDIA A100	**Jetson Nano (SBC)**
Application	PPE Detection	Face Mask	Crowd Counting	**Passenger Monitoring**

4.5 Limitations

This study validates the efficiency of the YOLOv8 architecture for real-time passenger detection in resource-constrained ferry terminals. However, some limitations must be noted. Firstly, the model was trained and validated using a custom dataset derived from the Maranhão ferry terminals surveillance system [23]. While this ensures high performance (97% precision) within that domain, its generalization to distinct environments (e.g., different camera angles or indoor lighting) may require fine-tuning [21].

Secondly, the current implementation focuses on object detection and latency (5.3 ms) but does not yet incorporate a robust Multi-Object Tracking (MOT) framework to handle continuous passenger trajectories [4]. Thirdly, although the Jetson Nano delivers production-ready latency, this performance comes at the cost of higher power consumption than CPU-bound Raspberry Pi devices, which may limit its suitability for purely battery-operated deployments [14, 15]. Finally, while the ONNX Runtime provided highly efficient results, maximal optimization via hardware-specific tools like NVIDIA TensorRT or model quantization was not explored in this study, representing a path for future industrial scaling [10, 15].

5 Conclusion

This research successfully developed and validated a robust Edge AI solution for real-time passenger monitoring in ferry terminals, demonstrating significant advancements in applying computer vision to transportation logistics [12, 23]. By following the CRISP-DM methodology, we established a systematic pipeline that culminated in the deployment of the YOLOv8-nano model, providing a reliable and scalable alternative to manual counting methods [21, 22].

The experimental results provided essential insights for systems engineering at the edge. First, the systematic dataset expansion to 2,229 images demonstrated that data diversity is paramount for model generalization, achieving a high **97% detection precision specifically for the person class** under production-like conditions [7, 27]. Second, the hardware benchmarking clarified the performance-resource boundaries of current SBCs. While CPU-bound platforms like the Raspberry Pi 3 and 4B proved inadequate for real-time demands, the **Jetson Nano emerged as the optimal archi-**

tecture, achieving an ultra-low latency of **5.3 ms per image** via GPU acceleration [14, 15].

Based on the limitations identified regarding generalization and power consumption, future research will focus on:

- **Model Optimization:** Implementing INT8 quantization and hardware-specific acceleration through the NVIDIA TensorRT framework to further reduce latency and energy footprint [10, 15].
- **Tracking Integration:** Incorporating Multi-Object Tracking (MOT) and Kalman filters to handle continuous passenger trajectories and reduce counting redundancies in dense flows [4, 24].
- **System Scalability:** Investigating Task Offloading mechanisms and Edge-Edge collaboration frameworks to distribute the computational load across multiple nodes during peak passenger traffic [1].

This work quantifies the essential trade-offs for Edge AI in critical public infrastructure, providing a foundational guide for implementing autonomous monitoring systems in maritime transportation [10, 23].

Acknowledgements We would like to thank EMAP for providing videos that enable us to build on the dataset. Additionally, this work utilized resources from the "Centro Nacional de Processamento de Alto Desempenho em São Paulo (CENAPAD-SP)".

Author contributions R.P.C.R. built the datasets, performed the experiments, and wrote the draft of the paper. O.A.C.C. conceived and supervised the project, advised the first author, elaborated the methodology, provided the single board for experiments, and write the final manuscript. Both authors reviewed the manuscript.

Funding The Article Processing Charge (APC) for the publication of this research was funded by the Coordenação de Aperfeiçoamento de Pessoal de Nível Superior - Brasil (CAPES) (ROR identifier: 00x0ma614). This study was financed in part by the Coordenação de Aperfeiçoamento de Pessoal de Nível Superior – Brasil (CAPES) – Finance Code 001 and Softex as TIC Residency 09 process 01245.005714/2022-18, conducted by the Instituto Federal do Maranhão (IFMA).

Data availability The public data is available upon request. The private one is subject to EMAP evaluation.

Declarations

Conflict of interest The authors declare no competing interests.

Open Access This article is licensed under a Creative Commons Attribution 4.0 International License, which permits use, sharing, adaptation, distribution and reproduction in any medium or format, as long as you give appropriate credit to the original author(s) and the source, provide a link to the Creative Commons licence, and indicate if changes were made. The images or other third party material in this article are included in the article's Creative Commons licence, unless indicated otherwise in a credit line to the material. If material is not included in the article's Creative Commons licence and your intended use is not permitted by statutory regulation or exceeds the permitted use, you will need to obtain permission directly from the copyright holder. To view a copy of this licence, visit http://creativecommons.org/licenses/by/4.0/.

References

1. Wu D, Li Z, Ma Y, et al (2025) Task offloading based on two types of edge-edge collaboration in mobile edge computing. Computing 107:74. https://doi.org/10.1007/s00607-025-01420-3
2. Collini E, Palesi L, Ipsaro A, Nesi P, Pantaleo G, Zhao W (2023) Flexible thermal camera solution for smart city people detection and counting. https://doi.org/10.1007/s11042-023-16374-x
3. Dias R, Figueiredo FAP (2023) Comparação de modelos yolov5 e yolov8 para detecção de imagens de Áreas rurais. https://doi.org/10.13140/RG.2.2.30587.90400
4. Banelas D, Petrakis EGM (2024) Motioninsights: real-time object tracking in streaming video. Mach Vis Appl 35:85. https://doi.org/10.1007/s00138-024-01570-y
5. Gupta J, Sharma S (2025) Sway: efficient pedestrian detection using sam-based walkable area segmentation and yolo. Int J Inf Technol 17:3811–3822. https://doi.org/10.1007/s41870-025-02596-9
6. Tashtoush A, Wang Y, Khasawneh MT (2025) Real-time object segmentation for laparoscopic cholecystectomy using YOLOv8. Neural Comput Appl 37:2697–2710. https://doi.org/10.1007/s00521-024-10713-1
7. Shyaa TAR, Hashim AA (2024) Enhancing real human detection and people counting using yolov8. BIO Web Conf 97:00061. https://doi.org/10.1051/bioconf/20249700061
8. Guo Z, Wang X, Zhang Z et al (2023) Persnake: a real-time pedestrian instance segmentation network using contour representation. Mach Vis Appl 34:78. https://doi.org/10.1007/s00138-023-01419-w
9. Liu J, Wang H, Wang J, He D, Xu R, Tang X (2024) Thermal infrared action recognition with two-stream shift graph convolutional network. Mach Vis Appl 35:65. https://doi.org/10.1007/s00138-024-01550-2
10. Zhang Z, Li W, Luo L (2025) Yolo-gsd: a real-time pedestrian detection algorithm based on yolov8 in dense environments. J Real-Time Image Proc 22:193. https://doi.org/10.1007/s11554-025-01771-2
11. Ivasić-Kos M, Kristo M, Pobar M (2019) Person detection in thermal videos using yolo. In: Proceedings of SAI Intelligent Systems Conference IntelliSys 2019: intelligent systems and applications, vol 2. Springer, pp 254–267
12. Sá PCA, Quezia A, Marcus C, Maciel AMA, Bastos-Filho C, Lima Júnior C (2024) Yolov8 para controle de produção pós-colheita e beneficiamento de frutos. Revista de Engenharia e Pesquisa Aplicada 9(1):115–122. https://doi.org/10.25286/repa.v9i1.2788. (**License CC BY-NC 4.0**)
13. Erlina T, Fikri M (2023) Yolo algorithm-based visitor detection system for small retail stores using single board computer. J Appl Eng Technol Sci 4(2):908–920
14. Aicha FB (2025) Optimized embedded ai: efficient implementation of cnns on esp32-cam for real-time image classification. Computing 107:206–222. https://doi.org/10.1007/s00607-025-01559-z
15. Mela JL, Sánchez CG (2025) Correction: Yolo-based power-efficient object detection on edge devices for usvs. J Real-Time Image Proc 22:195. https://doi.org/10.1007/s11554-025-01776-x
16. Al-qaness MAA, Abbasi AA, Fan H, Ibrahim RA, Alsamhi SH, Hawbani A (2021) An improved yolo-based road traffic monitoring system. Computing 103:211–230. https://doi.org/10.1007/s00607-020-00869-8
17. Hou C (2023) The application of human detection based on yolov5. High Sci Eng Technol CSIC 34:203–208
18. Tang F, Yang F, Tian X (2023) Long-distance person detection based on yolov7. Electronics 12(6):1502. https://doi.org/10.3390/electronics12061502
19. Flôres DS (2023) Detecção de epis através de uma cnn. Monografia (bacharelado em engenharia de computação), Universidade Federal do Rio Grande do Sul, Porto Alegre . https://lume.ufrgs.br/bitstream/handle/10183/272129/001188438.pdf?sequence=1
20. Luan F, Fan K, Xu X, Yang X, Chen J (2025) Cherry-yolo: enhanced real-time detection of cherry ripeness and defects with optimised yolov8. Computing 107:198–205. https://doi.org/10.1007/s00607-025-01558-0
21. Lee H, Lee J (2024) Tinycount: an efficient crowd counting network for intelligent surveillance. J Real-Time Image Proc 21:153. https://doi.org/10.1007/s11554-024-01531-8
22. Wirth R, Hipp J (2000) Crisp-dm: Towards a standard process model for data mining. In: Proceedings of the 4th International Conference on the practical applications of knowledge discovery and data mining, pp 29–39
23. Portuária AE, EM (2024) Imagem capturada no terminal de ferryboat. Imagem obtida a partir do sistema de monitoramento da EMAP

24. Motwani NPSS (2023) Human activities detection using deep learning technique- yolov8. ITM Web Conf 56:03003. https://doi.org/10.1051/itmconf/20235603003
25. Gao C, Zhao G, Gao S, Du S, Kim E, Shen T (2024) Advancing architectural heritage: precision decoding of east Asian timber structures from tang dynasty to traditional japan. Herit Sci 12(1):8. https://doi.org/10.1186/s40494-024-01332-z
26. Aicha FB (2025) Optimized embedded ai: Efficient implementation of cnns on esp32-cam for real-time image classification. Computing 107:206–222. https://doi.org/10.1007/s00607-025-01559-z
27. Andrade KGORd, Pereira YdS, Silveira CHPd, Meneses AAdM (2023) Uma análise do yolov8: Detecção de pessoas com máscaras faciais. In: Anais do XXVI ENMC – Encontro Nacional de Modelagem Computacional, XIV ECTM – Encontro de Ciência e Tecnologia de Materiais, Santarém, PA, Brasil (2023). Universidade Federal do Oeste do Pará (UFOPA)
28. Ferreira FRT, Couto LM, Melo Baptista Domingues G (2024) Exploring the potential of YOLOv8 in hybrid models for facial mask identification in diverse environments. Neural Comput Appl 36:22037–22052. https://doi.org/10.1007/s00521-024-10351-7

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.